

Homework

No Teams (Every student must make an individual submission. It must compile using make on Ubuntu).

- **(1) Calculate the average user rating per movie and append to the results from moviemarket. Take a screenshot for the zip file**
e.g. `moviemarket -title Las Vegas`
Would give
25::Leaving Las Vegas (1995)::Drama|Romance [rating]
1884::Fear and Loathing in Las Vegas (1998)::Adventure|Comedy|Crime|Drama|Fantasy [rating]
7420::Viva Las Vegas (1964)::Comedy|Musical|Romance [rating]
 - **(2) Write a program called RatingDistance** which should be an average of the distance between the ratings of the movies that the two users have seen together (ignore movies that either one have not rated). In the example in the slides, the RatingDistance between Terry and Jamie should be 2.5.
Command: `RatingDistance [UserID1] [UserID2]`
Output: `The RatingDistance is [Distance]`

In general, when comparing RatingDistance between two users, only use the ratings for a movie that BOTH users have seen. e.g

	M1	M2	M3
Blake	-	4	2
Logan	3	5	5
Jean	-	-	4

The RatingDistance between Blake and Logan would be 2 (1+3=4 divided by 2 movies)
The RatingDistance between Jean and Logan would be 1

Run your program for UserID1: 453 and UserID2: 10425, and take a screenshot of the terminal window for the zip file.
 - **(3) Write an MDC Basic Collaborative Filtering program called "MovieRate"** which reads in a user's movie ratings and then predicts a rating for an unseen movie. Input is a user ID and a movie ID and the output is the predicted rating with details.
 - Command: `MovieRate [UserID] [MovieID]`
 - Output: `The predicted rating for UserID: [UserID] and MovieID: [MovieID] is [PredictedRating] based on the most similar user: [SimilarUserID] with a ratingdistance: [Distance]`
 - e.g. (numbers not real) The predicted rating for UserID: 11 and MovieID: 222 is 3 based on the most similar user: 4444 with a ratingdistance: 2

Take a screenshot of your program working for UserID:13, MovieID:32 for the zip file. If there are multiple equally similar users, just take the one with the lowest ID number. Important: Explain your algorithm and what you do for non-easy cases (ratings sparsity)
- Put the answers in a zip file of a directory called your name (e.g. joost.kok, with subdirs P1, P2 and P3, one subdir per problem)
- A file in the main directory, answers.txt which has your assessment if you solved each problem and also specifies the LIACS computer room and computer name (see side of computer) where it worked
 - Each subdir contains your sourcecode with Makefile(s) (where appropriate) separating the problems is for easier grading by the TAs
 - **Do not put the MovieLens 10M dataset in the zip file** – e.g. Your program should assume that the original *.dat files are in the same directory as the executable (e.g. `fp = fopen("ratings.dat", "r");`)
 - `joost.kok`
 - `answers.txt`
 - `P1` (a directory containing the screenshot and code of the solution for Problem 1)
 - `P2` (containing the screenshot and code of the solution for Problem 2)
 - `P3` (containing the screenshot and code of the solution for Problem 3)
- Due date: see LCM (roughly one week, late penalties as mentioned in the slides)